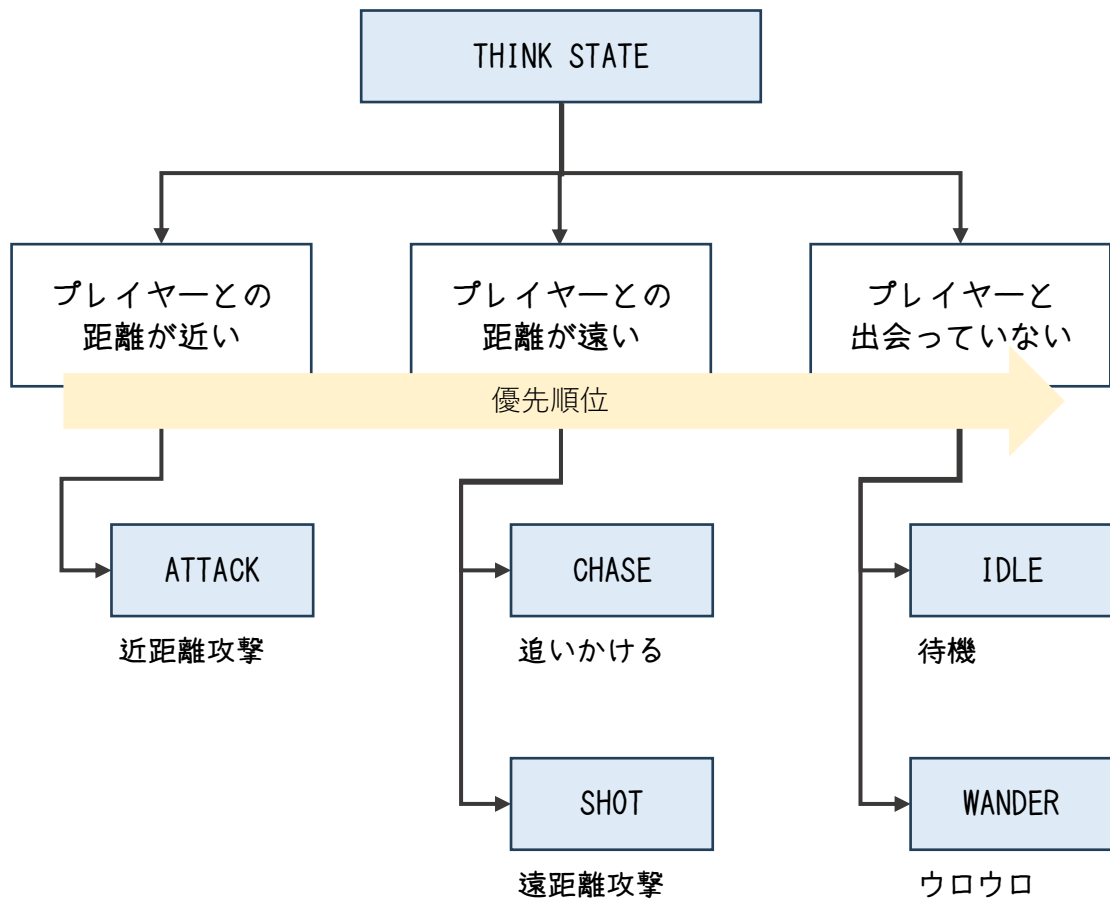


NPCの行動制御①(思考STATE)

NPCの行動プログラムを実装する際に、思考STATEを一般的に使用します。
下記はよくある状態遷移図です。



試しに、NPCのネズミに、
IDLEとWANDER状態を実装してみましょう。

EnemyBase.h

```
// アニメーション種別
enum class ANIM_TYPE
{
    IDLE = 9,
    WALK = 13,
};
```

EnemyBase.cpp

```
void EnemyRat::ChangeStateThink(void)
{

    stateUpdate_ = std::bind(&EnemyRat::UpdateThink, this);

    // 思考
    // ランダムに次の行動を決定
    // 30%で待機、70%で徘徊
    int rand = GetRand(100);
    if (rand < 30)
    {
        ChangeState(STATE::IDLE);
    }
    else
    {
        ChangeState(STATE::WANDER);
    }

}

void EnemyRat::ChangeStateIdle(void)
{

    stateUpdate_ = std::bind(&EnemyRat::UpdateIdle, this);

    // ランダムな待機時間
    step_ = 3.0f + static_cast<float>(GetRand(3));

    // 移動量ゼロ
    movePow_ = AsoUtility::VECTOR_ZERO;

    // 待機アニメーション再生
    animationController_>Play(
        static_cast<int>(ANIM_TYPE::IDLE), true);

}
```

```

void EnemyRat::ChangeStateWander(void)
{

    stateUpdate_ = std::bind(&EnemyRat::UpdateWander, this);

    // ランダムな角度
    float angle = static_cast<float>(GetRand(360)) * DX_PI_F / 180.0f;

    // 移動方向
    moveDir_ = VGet(cosf(angle), 0.0f, sinf(angle));

    // ランダムな移動時間
    step_ = 2.0f + static_cast<float>(GetRand(5));

    // 移動スピード
    moveSpeed_ = 3.0f;

    // 歩きアニメーション再生
    animationController_>Play(
        static_cast<int>(ANIM_TYPE::WALK), true);

}

```

【要件】

待機とウロウロをランダムに繰り返す、ネズミの行動制御を完成させること。

- UpdateIdleの完成

指定時間経過したら、思考STATEに戻る。

- UpdateWanderの完成

指定時間経過したら、思考STATEに戻る。

指定された移動方向と、指定された移動スピードを使用して、移動量(movePow_)を作る。

- InitAnimation

WALKアニメーションを追加。(再生スピード30くらい)

【目標】

ネズミが待機とウロウロを繰り返すこと。